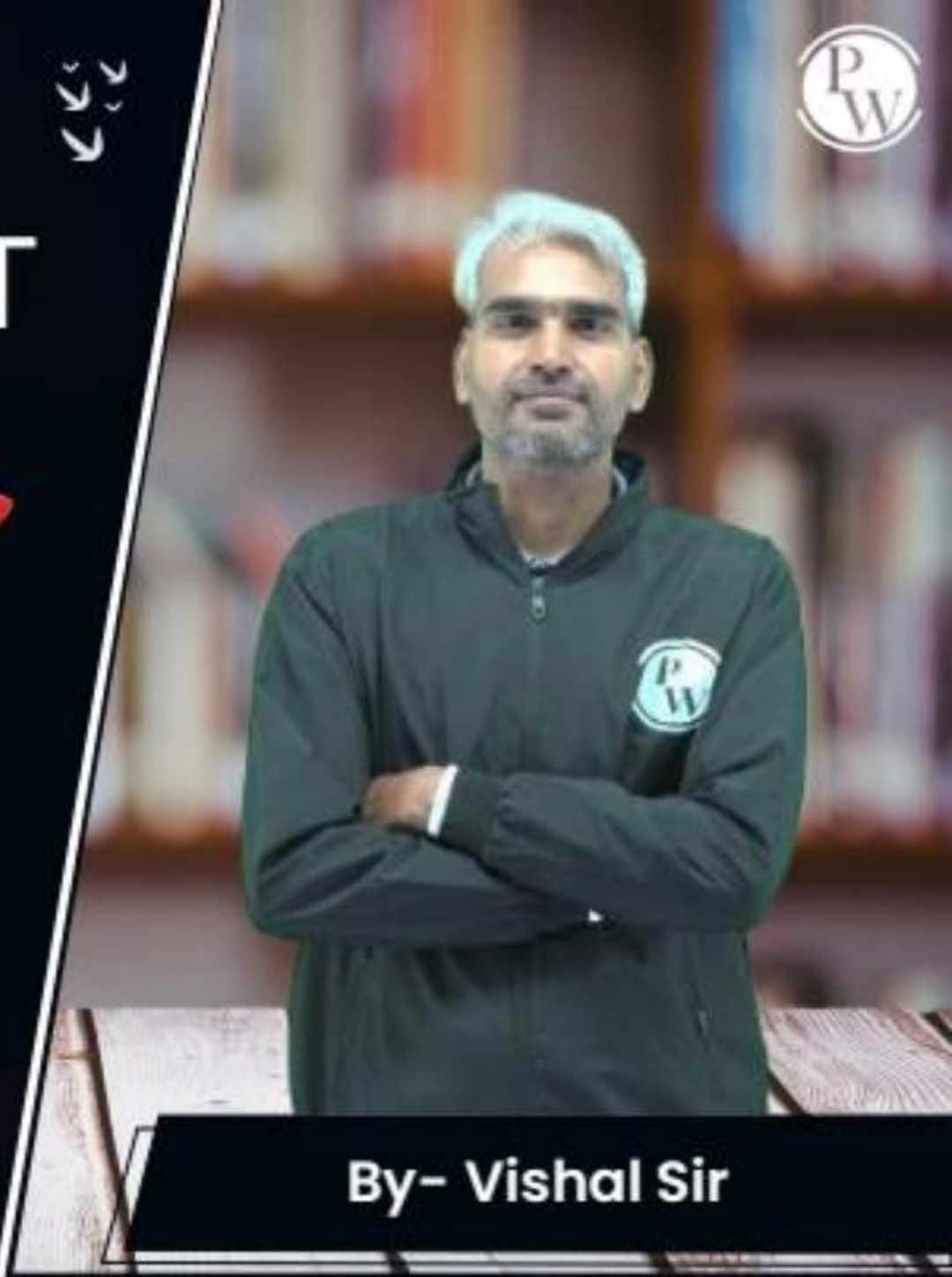
Computer Science & IT



Database Management System

Transactions and Concurrency control

Lecture No. 09



By- Vishal Sir

Recap of Previous Lecture



Topic Basic 2PL and lock upgrading/downgrading

Topic Problems possible with Basic 2PL

- { irrecoverability + Cascading Rollback } → Solu → Strict-2PL
  - { Uncommitted Read + Simultaneous Write-Write opn not allowed }
- Deadlock → Solu → Conservative 2PL
- Starvation → (No Solution)

Slide

Topics to be Covered



- Topic: Strict 2PL
- Topic: Conservative 2PL
- Topic: Rigorous 2PL
- Topic: Basic time stamp ordering protocol
- Topic: Thomas write time stamp ordering protocol



Topic : Classification of Schedules based on 2PL

Strict 2PL

Conservative 2PL

Rigorous 2PL



Topic : Strict 2PL

Basic 2PL + Strict Recoverability = Strict 2PL

In strict 2PL a transaction can unlock the locks only after the Commit operation of that transaction

↓

i.e. Exclusive locks must be held until Commit operation

Basic 2PL +

T_1	T_2
W(A)	
⋮	
Commit/Rollback	
	$R(A)/W(A)$

=

T_1	T_2
X(A)	
	⋮
Commit U(A)	
	$S(A)/X(A)$

Strict 2PL :-

1. Ensures serializability
2. Ensure recoverability
3. Avoid Cascading Rollback problem
4. No lost update problem

* But Strict 2PL may suffer from deadlock & Starvation



Topic : Conservative 2PL

* Conservative 2PL is used to avoid deadlock

T_1	T_2
S(A)	
R(A)	
	✓ X(B)
	W(B)
← holds lock on data item A & wait for lock on B	
denied S(B) / ✗ R(B)	
X(A) denied / ✗ W(A)	
← holds lock on data item B & Waiting for the lock on data item A.	

- We can avoid the deadlock by dis-satisfying the necessary Condn hold & wait.

Hold & wait can be dis-satisfied using hold or wait



Topic : Conservative 2PL

* Conservative 2PL is used to avoid deadlock
* In conservative 2PL a transaction will request for all the locks required for its execution, before starting the execution

If all the locks are granted then transaction will hold all the locks and start its execution

(or)

If any of the requested lock is not granted then transaction will release all the granted locks and wait for all the locks to become available at the same time

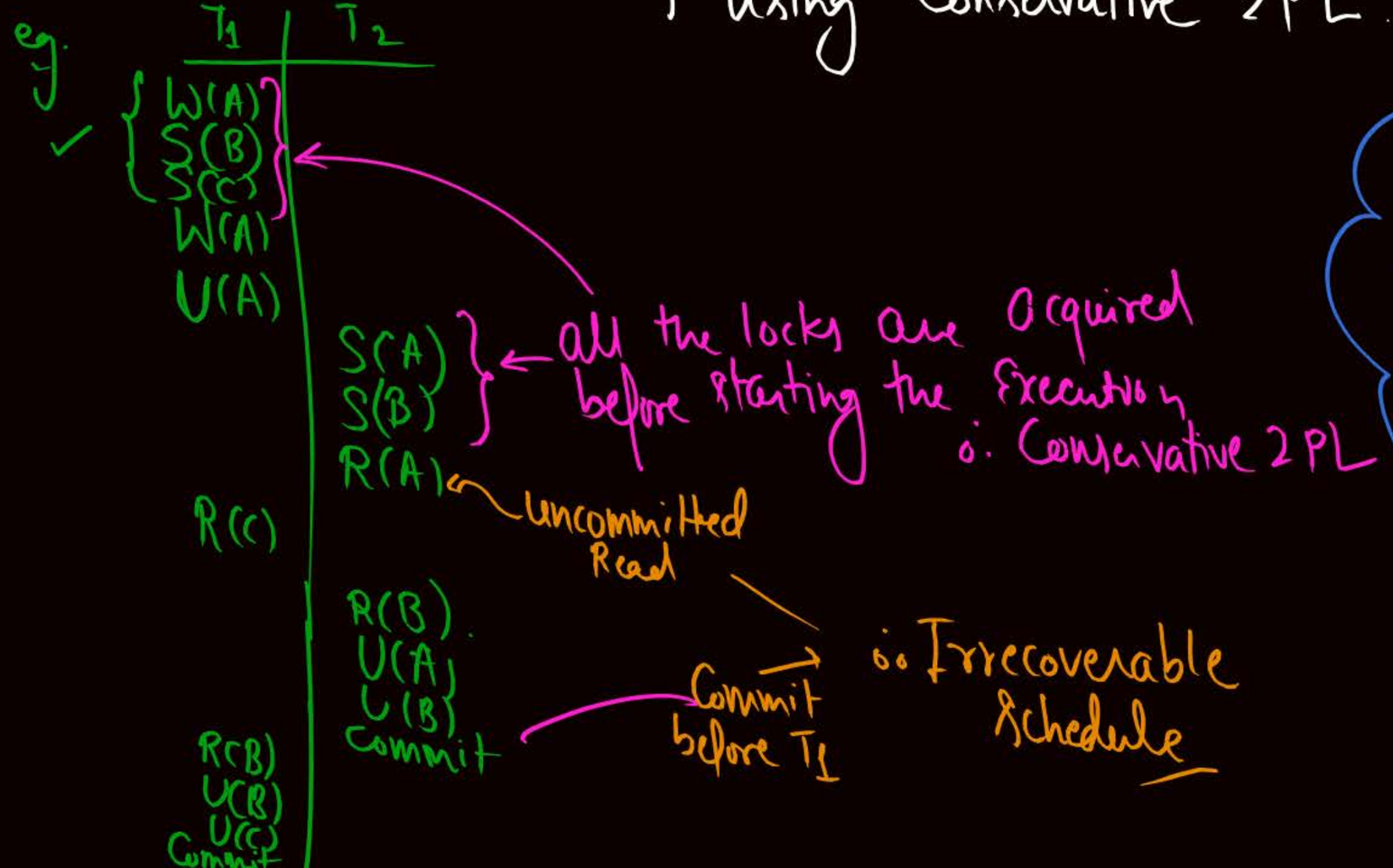
i.e. either the transaction will hold all the locks required for its execution (i.e No deadlock)

or

it will not hold any lock and wait for all (i.e. Not stopping others)

* Conservative 2PL does not define the order of unlock opn

∴ irrecoverability, lost-update, & cascading roll back are still possible (Apart from that Starvation is also possible using Conservative 2PL.)

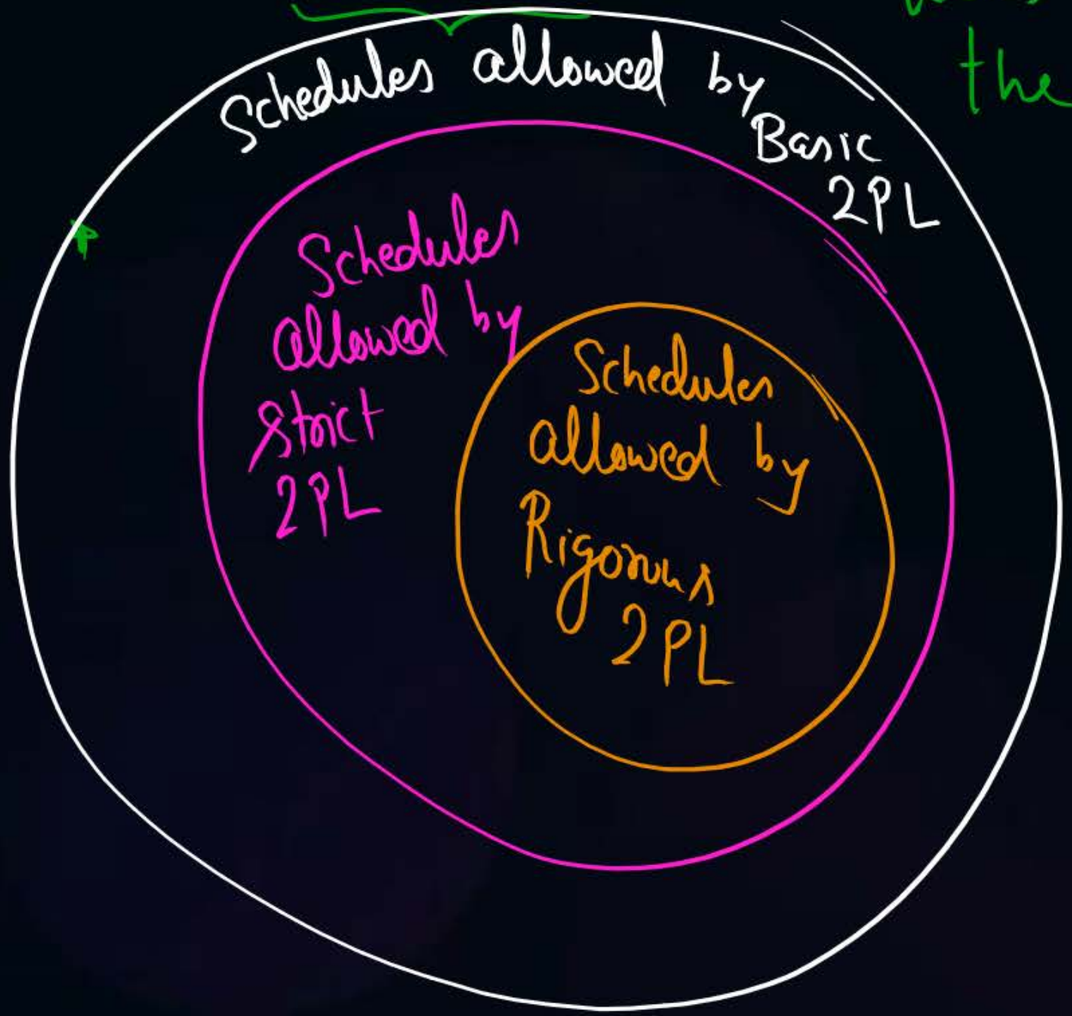


In Conservative 2PL we need to have the Prior information about the operations which are going to be performed by the transaction



Topic : Rigorous 2PL

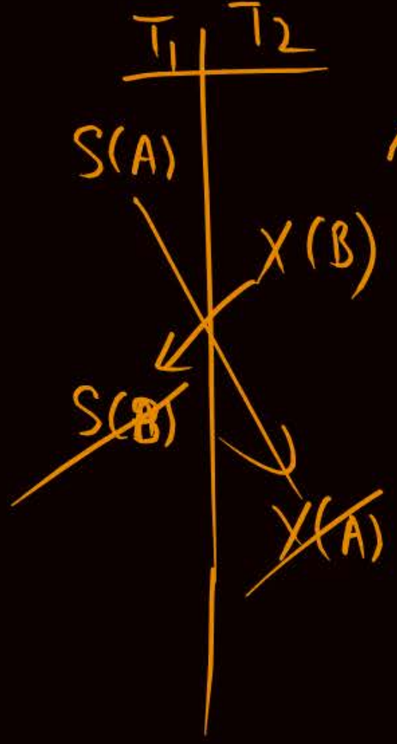
Basic 2PL + Both shared & Exclusive locks are held until the Commit operation = Rigorous 2PL



T_1	T_2
S(A)	
⋮	
X(B)	
⋮	
Commit	
U(A)	
U(B)	

all unlock opⁿ will be performed only after Commit

Rigorous - 2PL only defines the order of unlock {i.e. unlock after Commit}, but it does not define the order in which locks can be acquired, ∴ Deadlock is still possible



Implementation of Rigorous 2PL is easy because we need to unlock all the locks after the Commit



Topic : Time stamp ordering protocols

- Basic Time Stamp ordering protocol
- Thomas write time stamp ordering Protocol
  (Time stamp ordering Protocols with "Thomas Write" rule)

- **Time stamp:** Time stamp is a unique value assigned by DBMS to each transaction in ascending order

Let T_1 & T_2 are two transactions with time stamp values $TS(T_1)$ & $TS(T_2)$ respectively and if $TS(T_1) < TS(T_2)$

then T_1 is an old transaction & T_2 is younger transaction

Read Time Stamp of dataitem A :-

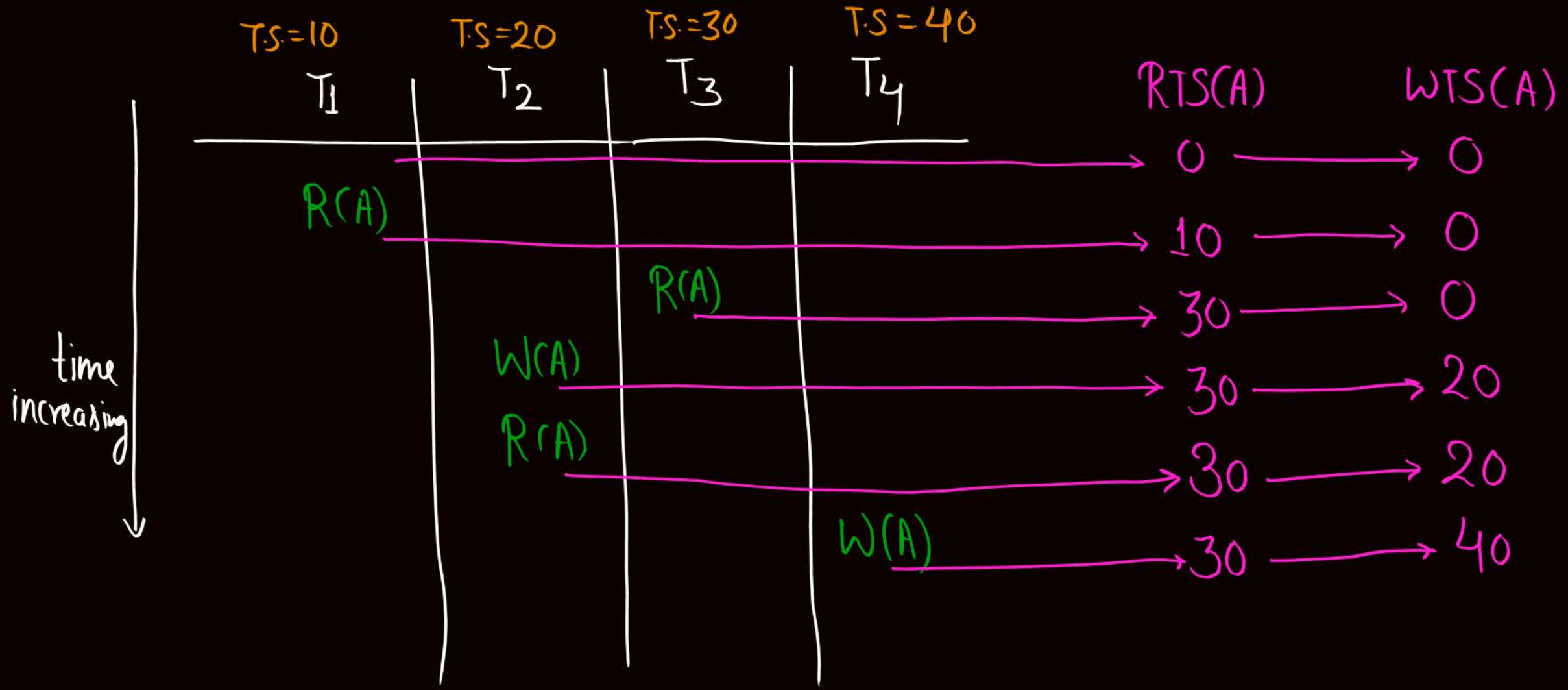
RTS(A) :- It is the highest value among the time stamps of the ongoing transactions that has performed the Read(A) opⁿ successfully

Write Time Stamp of dataitem A :-

WTS(A) :- It is the highest value among the time stamps of the ongoing transactions that has performed the Write(A) opⁿ successfully

RTS(A) & WTS(A)

initially RTS(A) = 0, WTS(A) = 0 } i.e. No transaction has performed Read or Write opn on data item A



Basic Time Stamp ordering Protocol :

A schedule is allowed to execute using B.T.S.O.P. only if schedule is Conflict serializable schedule and Conflict equivalent serial schedule base on time stamp ordering of the transactions.

eg: Consider the following schedule

T.S = 30 10 40 20

T_1	T_2	T_3	T_4

Time stamp order : $T_2 \rightarrow T_4 \rightarrow T_1 \rightarrow T_3$

This schedule will be allowed to execute using B.T.S.O.P. only if schedule is Conflict equivalent to serial schedule $T_2 \rightarrow T_4 \rightarrow T_1 \rightarrow T_3$

Basic Time Stamp ordering Protocol :

(i)

10	20
T_1	T_2
	R(A)
R(A)	

TS order : $T_1 \rightarrow T_2$

Precedence graph



∴ Topological order are $T_1 \rightarrow T_2$ & $T_2 \rightarrow T_1$

If T_1 performs this R(A) op^n at this point then also the schedule is Conflict equivalent to serial schedule $T_1 \rightarrow T_2$ (Same as Time stamp order)

∴ Transaction T_1 is allowed to perform this R(A) op^n

And RTS(A) will remain same

(ii)

10	20	30
T_1	T_2	T_1
	W(A)	

after Rollback

TS order = $T_1 \rightarrow T_2$

Rollback

R(A)

time

If we allow transaction T_1 to perform this R(A) op^n at this point, then behaviour or schedule will not be conflict equivalent to serial schedule based on Time stamp order $T_1 \rightarrow T_2$

then T_1 is not allowed to perform this R(A) op^n using B.T.S.O.P.

∴ Rollback T_1

{ WTS(A) > TS(T_1) ∴ Rollback T_1 }

Precedence graph



∴ Topological order = $T_2 \rightarrow T_1$

Basic Time Stamp ordering Protocol :

(iii)

10	20
T_1	T_2
	R(A)
W(A)	

TS order = $T_1 \rightarrow T_2$

Precedence graph



∴ Topological order

$T_2 \rightarrow T_1 \neq T_1 \rightarrow T_2$

Rollback

If T_1 performs this W(A) opⁿ at this point then the schedule is not Conflict equivalent to serial schedule $T_1 \rightarrow T_2$ (Time stamp order)

∴ T_1 is not allowed to perform this W(A) opⁿ, ∴ Rollback T_1

If (RTS(A) > TS(T_1)) then Rollback T_1

(iv)

10	20
T_1	T_2
	W(A)
W(A)	

T.S. order = $T_1 \rightarrow T_2$

Precedence graph



Topological order

$T_2 \rightarrow T_1 \neq T_1 \rightarrow T_2$

If we allow transaction T_1 to perform this W(A) opⁿ at this point, then behaviour or schedule will not be conflict equivalent to serial schedule based on Time stamp order $T_1 \rightarrow T_2$

then T_1 is not allowed to perform this W(A) opⁿ using B.T.S.O.P.

∴ Rollback T_1

If (WTS(A) > TS(T_1)) then Rollback T_1

Basic Time Stamp ordering Protocol Conditions :-

Let $TS(T_i) < TS(T_j)$ { i.e T_i is old & T_j is new
∴ Time Stamp order = $T_i \rightarrow T_j$ }

Case (i) When transaction T_i issue a Read(A) op^n :-

ⓐ If $(WTS(A) > TS(T_i))$, then T_i is not allowed to perform R(A) op^n ∴ Rollback T_i

ⓑ otherwise, { i.e. if $(WTS(A) \leq TS(T_i))$ }, then T_i is allowed to perform R(A) op^n,
∴ Perform R(A) op^n of transaction T_i
And set $RTS(A) = Max(RTS(A), TS(T_i))$

Basic Time Stamp ordering Protocol Conditions :-

Let $TS(T_i) < TS(T_j)$ { i.e T_i is old & T_j is new
∴ Time stamp order = $T_i \rightarrow T_j$ }

Case (ii) When transaction T_i issue a Write(A) op^n :-

ⓐ If $(RTS(A) > TS(T_i))$, then T_i is not allowed to perform W(A) op^n ∴ Rollback T_i

ⓑ If $(WTS(A) > TS(T_i))$ then also T_i is not allowed to perform W(A) op^n ∴ Rollback T_i

ⓒ Otherwise T_i is allowed to perform W(A) op^n
∴ Perform W(A) op^n of transaction T_i
and Set, $WTS(A) = TS(T_i)$



2 mins Summary

- Topic: Strict 2PL
- Topic: Conservative 2PL
- Topic: Rigorous 2PL
- Topic: Basic time stamp ordering protocol
- Topic: Thomas write time stamp ordering protocol

Slide

THANK - YOU